

Gradient descent

Gradient descent is a method for unconstrained mathematical optimization. It is a first-order iterative algorithm for minimizing a differentiable multivariate function.

The idea is to take repeated steps in the opposite direction of the gradient (or approximate gradient) of the function at the current point, because this is the direction of steepest descent. Conversely, stepping in the direction of the gradient will lead to a trajectory that maximizes that function; the procedure is then known as *gradient ascent*. It is particularly useful in machine learning and artificial intelligence for minimizing the cost or loss function.^[1] Gradient descent should not be confused with local search algorithms, although both are iterative methods for optimization.

Gradient descent is generally attributed to Augustin-Louis Cauchy, who first suggested it in 1847.^[2] Jacques Hadamard independently proposed a similar method in 1907.^{[3][4]} Its convergence properties for non-linear optimization problems were first studied by Haskell Curry in 1944,^[5] with the method becoming increasingly well-studied and used in the following decades.^{[6][7]}

A simple extension of gradient descent, stochastic gradient descent, serves as the most basic algorithm used for training most deep networks today.

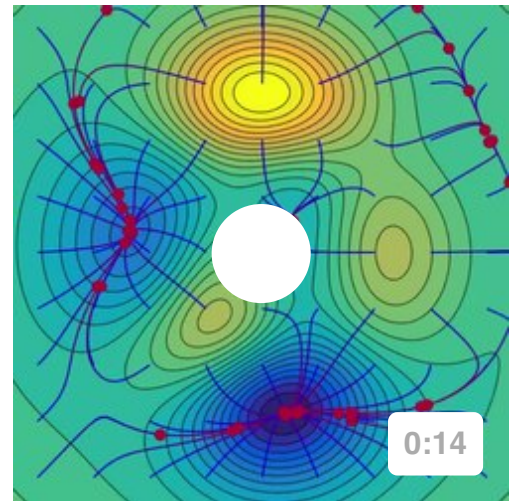
Description

Gradient descent is based on the observation that if the multi-variable function $f(\mathbf{x})$ is defined and differentiable in a neighborhood of a point $\mathbf{a}$, then $f(\mathbf{x})$ decreases *fastest* if one goes from $\mathbf{a}$ in the direction of the negative gradient of f at $\mathbf{a}$, $-\nabla f(\mathbf{a})$. It follows that, if

$$\mathbf{a}_{n+1} = \mathbf{a}_n - \eta \nabla f(\mathbf{a}_n)$$

for a small enough step size or learning rate $\eta \in \mathbb{R}_+$, then $f(\mathbf{a}_n) \geq f(\mathbf{a}_{n+1})$. In other words, the term $\eta \nabla f(\mathbf{a})$ is subtracted from $\mathbf{a}$ because we want to move against the gradient, toward the local minimum. With this observation in mind, one starts with a guess $\mathbf{x}_0$ for a local minimum of f , and considers the sequence $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \dots$ such that

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \eta_n \nabla f(\mathbf{x}_n), \quad n \geq 0.$$



Gradient descent in 2D

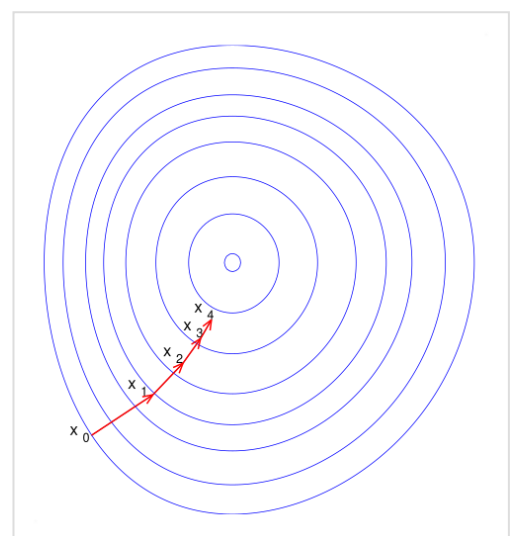


Illustration of gradient descent on a series of level sets

We have a monotonic sequence

$$f(\mathbf{x}_0) \geq f(\mathbf{x}_1) \geq f(\mathbf{x}_2) \geq \dots,$$

so the sequence $(\mathbf{x}_n)$ converges to the desired local minimum. Note that the value of the *step size* η is allowed to change at every iteration.

It is possible to guarantee the convergence to a local minimum under certain assumptions on the function f (for example, f convex and ∇f Lipschitz) and particular choices of η . Those include the sequence

$$\eta_n = \frac{|(\mathbf{x}_n - \mathbf{x}_{n-1})^\top [\nabla f(\mathbf{x}_n) - \nabla f(\mathbf{x}_{n-1})]|}{\|\nabla f(\mathbf{x}_n) - \nabla f(\mathbf{x}_{n-1})\|^2}$$

as in the Barzilai-Borwein method,^{[8][9]} or a sequence η_n satisfying the Wolfe conditions (which can be found by using line search). When the function f is convex, all local minima are also global minima, so in this case gradient descent can converge to the global solution.

This process is illustrated in the adjacent picture. Here, f is assumed to be defined on the plane, and that its graph has a bowl shape. The blue curves are the contour lines, that is, the regions on which the value of f is constant. A red arrow originating at a point shows the direction of the negative gradient at that point. Note that the (negative) gradient at a point is orthogonal to the contour line going through that point. We see that gradient *descent* leads us to the bottom of the bowl, that is, to the point where the value of the function f is minimal.

An analogy for understanding gradient descent

The basic intuition behind gradient descent can be illustrated by a hypothetical scenario. People are stuck in the mountains and are trying to get down (i.e., trying to find the global minimum). There is heavy fog such that visibility is extremely low. Therefore, the path down the mountain is not visible, so they must use local information to find the minimum. They can use the method of gradient descent, which involves looking at the steepness of the hill at their current position, then proceeding in the direction with the steepest descent (i.e., downhill). If they were trying to find the top of the mountain (i.e., the maximum), then they would proceed in the direction of steepest ascent (i.e., uphill). Using this method, they would eventually find their way down the mountain or possibly get stuck in some hole (i.e., local minimum or saddle point), like a mountain lake. However, assume also that the steepness of the hill is not immediately obvious with simple observation, but rather it requires a sophisticated instrument to measure, which the people happen to have at that moment. It takes quite some time to measure the steepness of the hill with the instrument. Thus, they should minimize their use of the instrument if they want to get down the mountain before sunset. The difficulty then is choosing the frequency at which they should measure the steepness of the hill so as not to go off track.



Fog in the mountains

In this analogy, the people represent the algorithm, and the path taken down the mountain represents the sequence of parameter settings that the algorithm will explore. The steepness of the hill represents the slope of the function at that point. The instrument used to measure steepness is

differentiation. The direction they choose to travel in aligns with the gradient of the function at that point. The amount of time they travel before taking another measurement is the step size.

Choosing the step size and descent direction

Since using a step size η that is too small would slow convergence, and a η too large would lead to overshoot and divergence, finding a good setting of η is an important practical problem. Philip Wolfe also advocated using "clever choices of the [descent] direction" in practice.^[10] While using a direction that deviates from the steepest descent direction may seem counter-intuitive, the idea is that the smaller slope may be compensated for by being sustained over a much longer distance.

To reason about this mathematically, consider a direction $\mathbf{p}_n$ and step size η_n and consider the more general update:

$$\mathbf{a}_{n+1} = \mathbf{a}_n - \eta_n \mathbf{p}_n.$$

Finding good settings of $\mathbf{p}_n$ and η_n requires some thought. First of all, we would like the update direction to point downhill. Mathematically, letting θ_n denote the angle between $-\nabla f(\mathbf{a}_n)$ and $\mathbf{p}_n$, this requires that $\cos \theta_n > 0$. To say more, we need more information about the objective function that we are optimising. Under the fairly weak assumption that f is continuously differentiable, we may prove that:^[11]

$$f(\mathbf{a}_{n+1}) \leq f(\mathbf{a}_n) - \eta_n \|\nabla f(\mathbf{a}_n)\|_2 \|\mathbf{p}_n\|_2 \left(\cos \theta_n - \max_{t \in [0,1]} \frac{\|\nabla f(\mathbf{a}_n - t\eta_n \mathbf{p}_n) - \nabla f(\mathbf{a}_n)\|_2}{\|\nabla f(\mathbf{a}_n)\|_2} \right) \quad (1)$$

This inequality implies that the amount by which we can be sure the function f is decreased depends on a trade off between the two terms in square brackets. The first term in square brackets measures the angle between the descent direction and the negative gradient. The second term measures how quickly the gradient changes along the descent direction.

In principle inequality (1) could be optimized over $\mathbf{p}_n$ and η_n to choose an optimal step size and direction. The problem is that evaluating the second term in square brackets requires evaluating $\nabla f(\mathbf{a}_n - t\eta_n \mathbf{p}_n)$, and extra gradient evaluations are generally expensive and undesirable. Some ways around this problem are:

- Forgo the benefits of a clever descent direction by setting $\mathbf{p}_n = \nabla f(\mathbf{a}_n)$, and use line search to find a suitable step-size γ_n , such as one that satisfies the Wolfe conditions. A more economic way of choosing learning rates is backtracking line search, a method that has both good theoretical guarantees and experimental results. Note that one does not need to choose $\mathbf{p}_n$ to be the gradient; any direction that has positive inner product with the gradient will result in a reduction of the function value (for a sufficiently small value of η_n).
- Assuming that f is twice-differentiable, use its Hessian $\nabla^2 f$ to estimate $\|\nabla f(\mathbf{a}_n - t\eta_n \mathbf{p}_n) - \nabla f(\mathbf{a}_n)\|_2 \approx \|t\eta_n \nabla^2 f(\mathbf{a}_n) \mathbf{p}_n\|$. Then choose $\mathbf{p}_n$ and η_n by optimising inequality (1).
- Assuming that ∇f is Lipschitz, use its Lipschitz constant L to bound $\|\nabla f(\mathbf{a}_n - t\eta_n \mathbf{p}_n) - \nabla f(\mathbf{a}_n)\|_2 \leq Lt\eta_n \|\mathbf{p}_n\|$. Then choose $\mathbf{p}_n$ and η_n by optimising inequality (1).
- Build a custom model of $\max_{t \in [0,1]} \frac{\|\nabla f(\mathbf{a}_n - t\eta_n \mathbf{p}_n) - \nabla f(\mathbf{a}_n)\|_2}{\|\nabla f(\mathbf{a}_n)\|_2}$ for f . Then choose $\mathbf{p}_n$ and η_n by optimising inequality (1).

- Under stronger assumptions on the function f such as convexity, more advanced techniques may be possible.

Usually by following one of the recipes above, convergence to a local minimum can be guaranteed. When the function f is convex, all local minima are also global minima, so in this case gradient descent can converge to the global solution.

Solution of a linear system

Gradient descent can be used to solve a system of linear equations

$$\mathbf{Ax} - \mathbf{b} = 0$$

reformulated as a quadratic minimization problem. If the system matrix $\mathbf{A}$ is real symmetric and positive-definite, an objective function is defined as the quadratic function, with minimization of

$$f(\mathbf{x}) = \mathbf{x}^\top \mathbf{Ax} - 2\mathbf{x}^\top \mathbf{b},$$

so that

$$\nabla f(\mathbf{x}) = 2(\mathbf{Ax} - \mathbf{b}).$$

For a general real matrix $\mathbf{A}$, linear least squares define

$$f(\mathbf{x}) = \|\mathbf{Ax} - \mathbf{b}\|^2.$$

In traditional linear least squares for real $\mathbf{A}$ and $\mathbf{b}$ the Euclidean norm is used, in which case

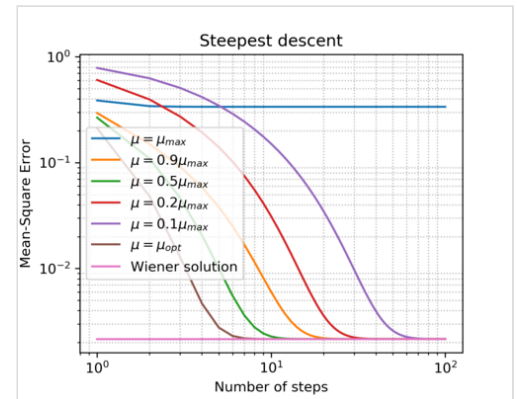
$$\nabla f(\mathbf{x}) = 2\mathbf{A}^\top (\mathbf{Ax} - \mathbf{b}).$$

The line search minimization, finding the locally optimal step size η on every iteration, can be performed analytically for quadratic functions, and explicit formulas for the locally optimal η are known.^{[6][13]}

For example, for real symmetric and positive-definite matrix $\mathbf{A}$, a simple algorithm can be as follows,^[6]

```
repeat in the loop:
     $\mathbf{r} := \mathbf{b} - \mathbf{Ax}$ 
     $\eta := \mathbf{r}^\top \mathbf{r} / \mathbf{r}^\top \mathbf{Ar}$ 
     $\mathbf{x} := \mathbf{x} + \eta \mathbf{r}$ 
    if  $\mathbf{r}^\top \mathbf{r}$  is sufficiently small, then exit loop
end repeat loop
return  $\mathbf{x}$  as the result
```

To avoid multiplying by $\mathbf{A}$ twice per iteration, we note that $\mathbf{x} := \mathbf{x} + \eta \mathbf{r}$ implies $\mathbf{r} := \mathbf{r} - \eta \mathbf{Ar}$, which gives the traditional algorithm,^[14]



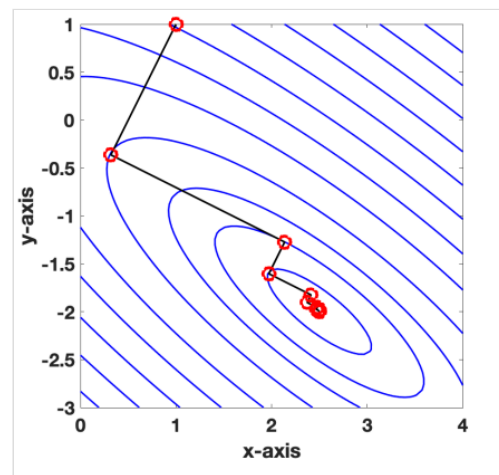
The steepest descent algorithm applied to the Wiener filter^[12]

```

r := b − Ax
repeat in the loop:
     $\eta := \mathbf{r}^\top \mathbf{r} / \mathbf{r}^\top \mathbf{A} \mathbf{r}$ 
    x := x +  $\eta \mathbf{r}$ 
    if  $\mathbf{r}^\top \mathbf{r}$  is sufficiently small, then exit loop
    r := r −  $\eta \mathbf{A} \mathbf{r}$ 
end repeat loop
return x as the result

```

The method is rarely used for solving linear equations, with the conjugate gradient method being one of the most popular alternatives. The number of gradient descent iterations is commonly proportional to the spectral condition number $\kappa(\mathbf{A})$ of the system matrix $\mathbf{A}$ (the ratio of the maximum to minimum eigenvalues of $\mathbf{A}^\top \mathbf{A}$), while the convergence of conjugate gradient method is typically determined by a square root of the condition number, i.e., is much faster. Both methods can benefit from preconditioning, where gradient descent may require less assumptions on the preconditioner.^[14]



Convergence path of steepest descent method for $\mathbf{A} = \begin{bmatrix} 2 & 2 \\ 2 & 3 \end{bmatrix}$

Geometric behavior and residual orthogonality

In steepest descent applied to solving $\mathbf{A}\mathbf{x} = \mathbf{b}$, where $\mathbf{A}$ is symmetric positive-definite, the residual vectors $\mathbf{r}_k = \mathbf{b} - \mathbf{A}\mathbf{x}_k$ are orthogonal across iterations:

$$\langle \mathbf{r}_{k+1}, \mathbf{r}_k \rangle = 0.$$

Because each step is taken in the steepest direction, steepest-descent steps alternate between directions aligned with the extreme axes of the elongated level sets. When $\kappa(\mathbf{A})$ is large, this produces a characteristic zig-zag path. The poor conditioning of $\mathbf{A}$ is the primary cause of the slow convergence, and orthogonality of successive residuals reinforces this alternation.

As shown in the image on the right, steepest descent converges slowly due to the high condition number of $\mathbf{A}$, and the orthogonality of residuals forces each new direction to undo the overshoot from the previous step. The result is a path that zigzags toward the solution. This inefficiency is one reason conjugate gradient or preconditioning methods are preferred.^[15]

Solution of a non-linear system

Gradient descent can also be used to solve a system of nonlinear equations. Below is an example that shows how to use the gradient descent to solve for three unknown variables, x_1 , x_2 , and x_3 . This example shows one iteration of the gradient descent.

Consider the nonlinear system of equations

$$\begin{cases} 3x_1 - \cos(x_2 x_3) - \frac{3}{2} = 0 \\ 4x_1^2 - 625x_2^2 + 2x_2 - 1 = 0 \\ \exp(-x_1 x_2) + 20x_3 + \frac{10\pi - 3}{3} = 0 \end{cases}$$